

VR Archery Game Features and Implementation Logic

1. VRShoot Component (Hand Colliders)

1.1 Overview

Purpose

Adds precise spherical colliders to the VR character's left and right hands to identify each hand's touch interaction with the bowstring (Notch). The native VR hand models have no precise collision usable for Overlap detection and cannot distinguish which hand touched the bowstring, so spherical colliders are dynamically created at runtime with a custom Object Type.

Core design

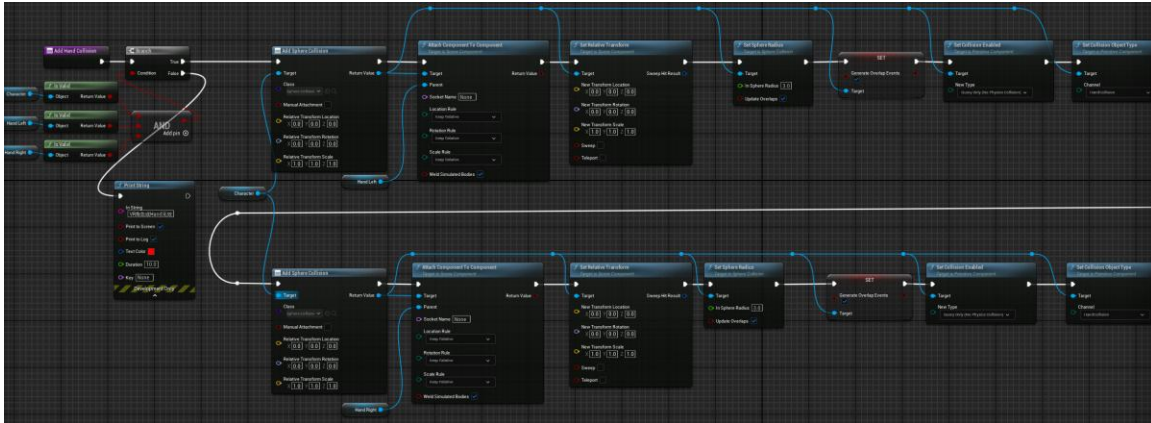
- Each hand gets a Sphere Collision component attached to its hand bone
- The collision Object Type is the custom Hand Collision, allowing precise filtering on the bowstring side
- The collider radius is independently configurable (left 3.0 / right 3.3) to fit the hands' different sizes
- Generate Overlap Events is enabled so the bowstring's Begin Overlap event can fire

1.2 Add Hand Collision Function

Purpose

At game initialization, dynamically creates one spherical collider for each of the VR character's hands and completes the attaching, radius setup, and collision configuration so the hands can be identified by the bowstring's Overlap. This is the physical foundation of the entire archery interaction — without hand colliders the bowstring cannot sense "which hand" is touching it.

Blueprint screenshots



Key points

- Collision is Query Only (query, no physics simulation): the hand colliders are only for Overlap detection; enabling simulation would conflict with the VR hands' skeletal animation and let the physics engine disturb the hands
- Setting the Object Type to the custom Hand Collision channel is the key: the bowstring's Notch enables an Overlap response for this channel in Project Settings, so the bowstring only responds to "hand" touches and nothing else in the scene

2. BP_Weapon_Bow (Bow)

2.1 Overview

Purpose

Implements the VR archery game's core weapon — the bow. The player holds the bow with one hand; when the other hand touches the bowstring (Notch) an arrow is spawned, and while the grip button is held the bowstring is drawn (the string model and animation follow in sync); on release the force is computed from the draw distance and the arrow is fired. The bow inherits from BP_Weapon_Base, reusing its grabbing system and adding bowstring interaction, draw animation, and arrow launching.

Core variables

Variable	Type	Description
Character	BP_MyVRCharacter	VR character reference, used to access the VRShoot component
VRshoot	VRShoot component	The character's VRShoot component reference, providing the hand colliders
Anim Bow	ABP_Bow	The bow's animation instance reference, controlling the draw

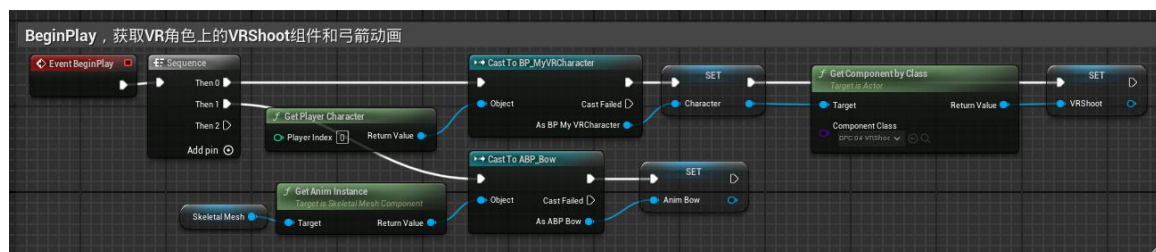
		animation progress
Is Pickup?	Boolean	Whether the bow is held by the player
Hold Motion Controller	Motion Controller	The motion controller of the hand holding the bow
Hold Hand	Scene Component	The bone of the hand holding the bow
Is Right Hand Hold?	Boolean	Whether the right hand holds the bow (the drawing hand is then the left)
Is Notch Touching?	Boolean	Whether the bowstring is being touched by the other hand (drawing ready state)
Arrow Motion Controller	Motion Controller	The motion controller of the drawing hand
Arrow Hand	Skeletal Mesh Component	The skeletal mesh component of the drawing hand
Arrow	BP_Arrow	Reference to the currently spawned arrow
Distance	Float	Current draw distance (0 ~ MaxDistance)
Max Distance	Float	Maximum draw distance (decides the maximum launch speed and full-draw state)
Will Shoot?	Boolean	Whether fully drawn (ready to fire)

2.2 BeginPlay — Initialization

Purpose

At game start, completes the bow's initialization: gets the VR character and its VRShoot component (for later hand collider interaction) and the bow's own animation instance (for controlling the draw animation).

Blueprint screenshots



2.3 Grab and Drop

Purpose

Handles the bow's two key lifecycle events, pickup and drop. On pickup, records the holding hand and determines left or right, preparing for the later bowstring interaction and shooting; on drop, restores the drawing hand to the character, plays the animation in reverse, destroys the bow, and notifies the GameManager to check the game end state and weapon consumption count.

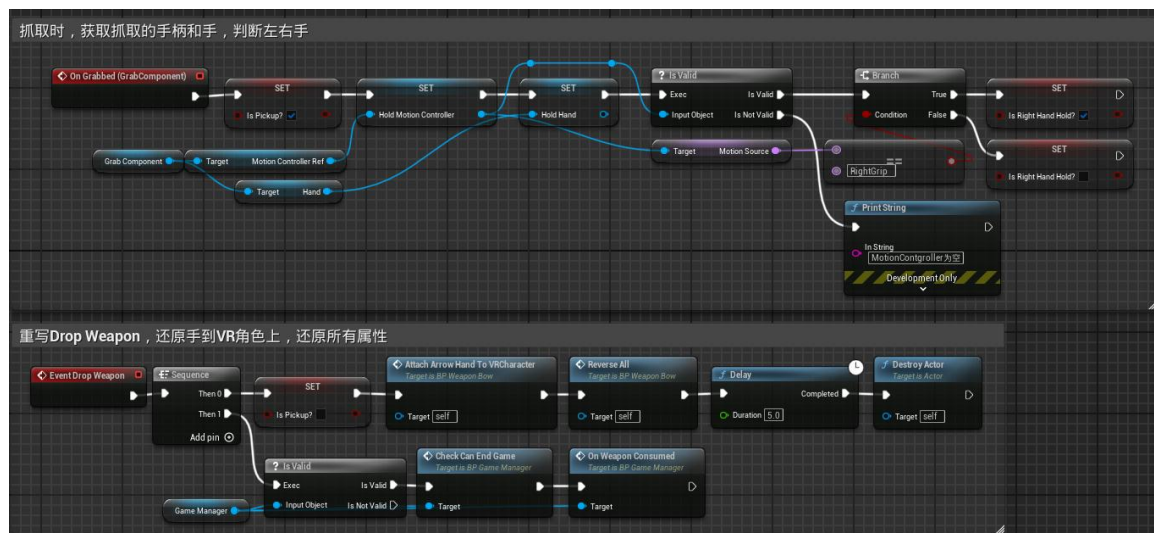
OnGrabbed (grab event)

Fires when the player grabs the bow; records the holding info and determines the hand.

Event Drop Weapon (drop event override)

Fires when the player drops the bow; restores the hand's position and destroys the bow.

Blueprint screenshots



Key points

- Is Right Hand Hold? is decided by comparing the Motion Source string with "RightGrip": this boolean is the switch for all later left/right branches (drawing-hand lookup, Socket selection, and input reading all depend on it)
- On drop, the arrow hand must be reattached to the character before destroying the bow: destroying it directly makes the hand bone attached to the bow vanish too, causing the severe bug of the VR character missing a hand

2.4 Bowstring Touch — Spawning the Arrow

Purpose

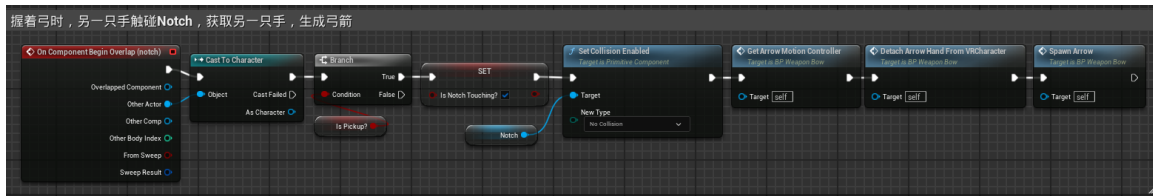
When the other hand touches the bowstring, the "nock arrow" flow triggers: mark the bowstring touch state, disable the Notch collision to prevent repeated triggering, find the drawing hand's controller and hand bone, detach the hand from the character and attach it to the bow (so the hand follows the bowstring), and finally spawn and attach an arrow

at the arrow hand's Socket. This is the entry from "holding the bow" into the "drawing ready" state.

Trigger conditions

Fires when the bow is held (Is Pickup? = true) and the other hand's collider touches the bowstring (Notch). The Notch is a collider component on the bow representing the bowstring's touch area.

Blueprint screenshots



Key points

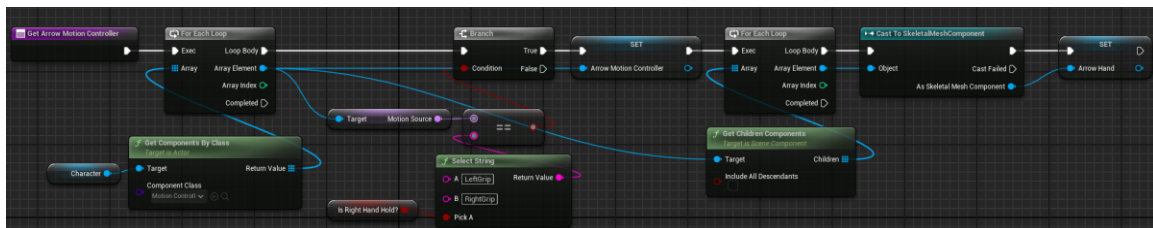
- The Is Pickup? check is the gate: touching the bowstring only counts while the bow is held; with the bow on the ground, a hand touching the string does not nock an arrow
- Disabling the Notch collision is the core of preventing repeats: the Overlap event can fire every frame, and not disabling it immediately would spawn multiple arrows within one frame; the collision is restored later in Attack or CheckHandClose
- The trigger order matters: find the drawing hand → detach the hand → spawn the arrow; reversing the order leaves the arrow with no attach target

2.5 Get Arrow Motion Controller — Finding the Drawing Hand

Description

Finds the other hand's (the drawing hand's) motion controller and hand bone based on "which hand holds the bow". Core idea: iterate all of the character's Motion Controllers and find the one opposite the holding hand.

Blueprint screenshots



Key points

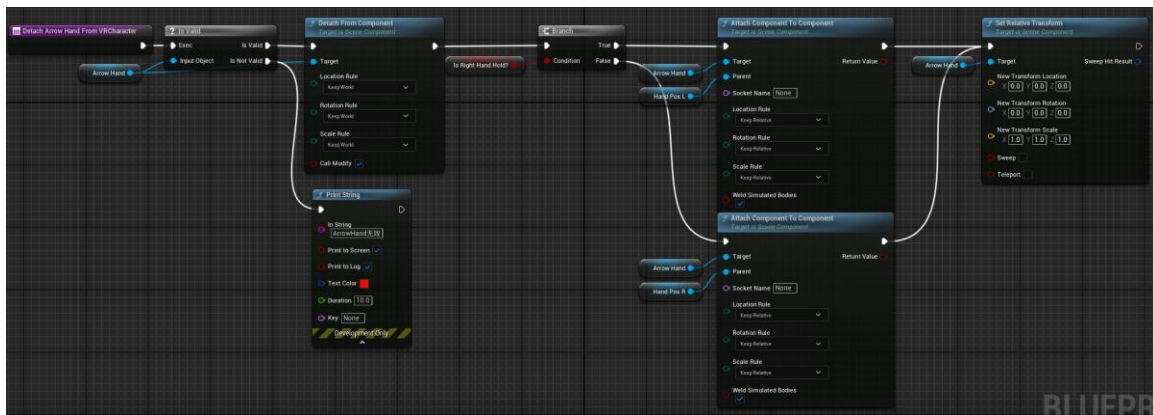
- Select String achieves dynamic target selection: Is Right Hand Hold? outputs "LeftGrip" when true and "RightGrip" when false, avoiding a duplicated structure of two Branches iterating separately
- The hand bone is a child of the Motion Controller: find it downward via Get Children Components and a Cast — the standard path for "locating the hand model from the controller"

2.6 Detach Arrow Hand From VRCharacter — Detaching the Hand onto the Bow

Description

Detaches the drawing hand's bone from the VR character and attaches it to the matching Hand Pos (Hand Pos L / Hand Pos R) on the bow. The visual effect: the hand "sticks" to the bowstring and moves with it, creating a realistic drawing feel.

Blueprint screenshots



Key points

- Hand Pos L / R are two preset scene component anchors on the bow: the positions the left/right hand should rest at while drawing
- Zeroing Set Relative Transform aligns the hand exactly with the anchor: without it the hand attaches with the relative offset it had when detached, landing in the wrong place

2.7 Spawn Arrow — Spawning the Arrow

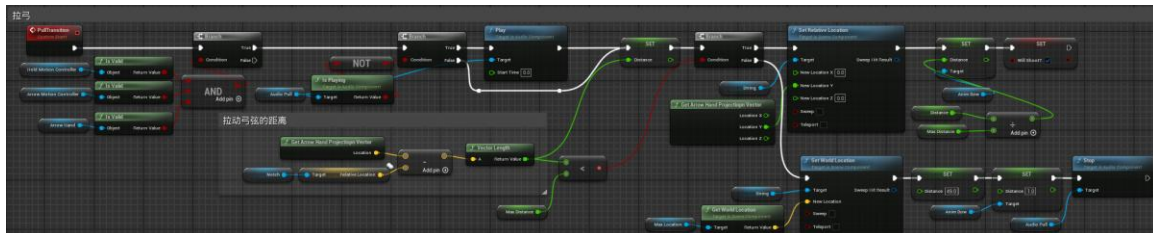
Description

Spawns a BP_Arrow at the drawing hand's designated Socket and attaches it. The arrow's initial position and rotation come from the arrow hand's world transform, ensuring the arrow sits correctly in the hand.

Blueprint screenshots

- Draw distance = clamp (|arrow hand projected position - Notch position| , 0 , MaxDistance)
- Animation parameter = Distance / MaxDistance (a 0–1 normalized value driving the animation Blueprint's draw progress)

Blueprint screenshots



Key points

- The projection vector is the core of the distance calculation: the arrow hand's position is projected into the Notch's local coordinate system and only the draw-direction component is taken, so hand wobble up or down does not affect the draw reading — a stable feel
- Min(Float) caps the distance: past MaxDistance the distance stops growing and the bowstring rests at its maximum, preventing over-drawing

2.10 Attack — Firing the Arrow

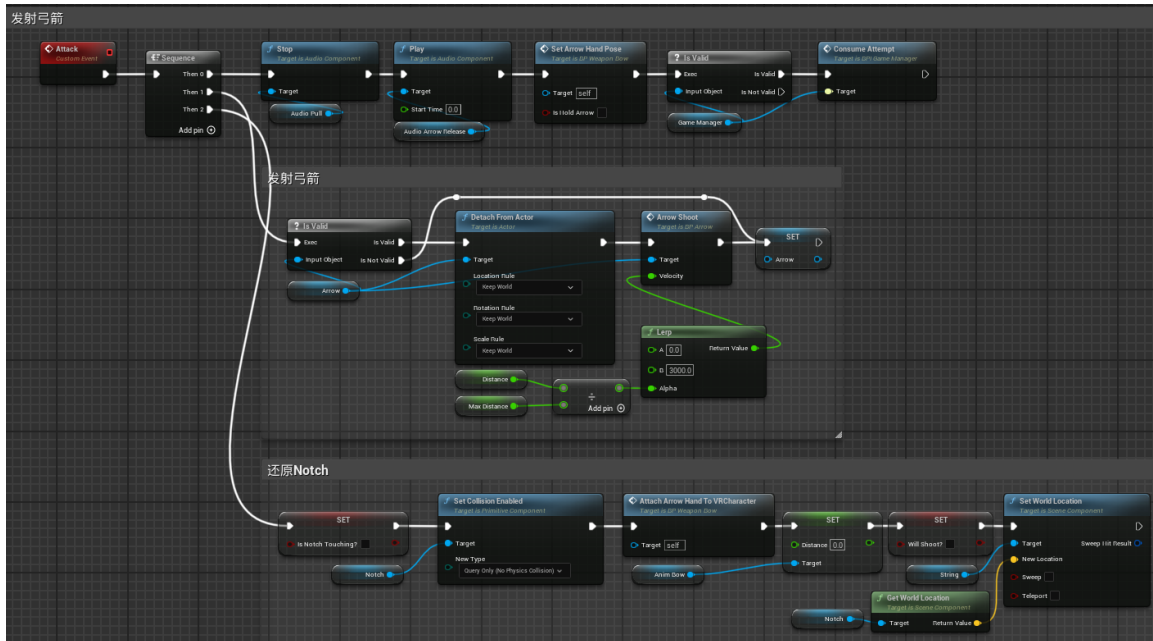
Description

Fires when the player releases the grip button: launches the arrow and restores the draw state. The launch force is decided by the draw distance — maximum speed (3000) at full draw, proportionally less when under-drawn.

Design notes

- Firing and scoring are two independent logics: the bow's ability to fire is never limited by attempts (it can always shoot); attempts only control whether points are awarded and whether the game ends
- Launch speed uses Lerp interpolation: 0 distance = 0 speed, full draw = 3000 speed, linearly interpolated in between

Blueprint screenshots



Key points

- Before firing, the arrow must Detach From Actor (Keep World): the arrow first leaves the hand bone keeping its world position, then flies driven by its own ProjectileMovement; without detaching, the arrow would keep following the hand
- The speed interpolation parameter is Distance/MaxDistance (0–1): it shares the same normalized value as the draw animation, so how far it visually drew is exactly how strong it is — consistent feedback
- SET Arrow = None is the key finishing touch: only after clearing the reference can the next nocked arrow be tracked correctly; otherwise CheckHandClose might destroy an already-fired arrow

2.11 Tick

Description

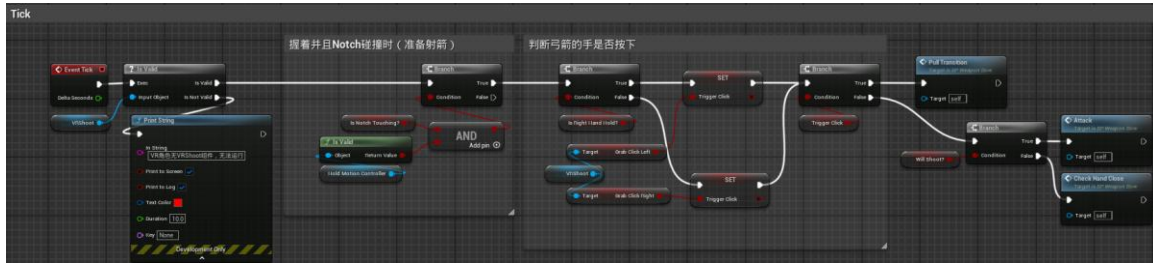
The bow's main loop scheduler, judging the current state and running the matching logic every frame. Only when the bowstring is touched and the hold is valid does it enter the drawing/firing flow.

State branch table

State condition	Grip button	Action
Bowstring touched + hold valid	Pressed + WillShoot=true	Attack — fire
Bowstring touched + hold valid	Pressed + WillShoot=false	CheckHandClose — detach detection
Bowstring touched + hold valid	Not pressed	PullTransition — draw following
Bowstring not touched or	-	No archery logic runs

hold invalid		
--------------	--	--

Blueprint screenshots



Key points

- Overlap only handles the "nock" (entering the state); the drawing, firing, and detach detection afterward all rely on the Tick event's per-frame polling
- Input reading selects dynamically by the holding hand: holding with the right hand reads "Grab Click Left" (left hand drawing), and vice versa reads Right

3. BP_Arrow (Arrow)

3.1 Overview

Purpose

Implements the fired arrow. The arrow flies driven by a `ProjectileMovementComponent` and stops and destroys on hitting something. The arrow inherits directly from `Actor` instead of `BP_Weapon_Projectile_Base` because the parent enables physics simulation, which conflicts with `ProjectileMovement` and prevents the arrow from flying. As a plain `Actor` with only `ProjectileMovement`, flight works correctly.

Core components

- **Projectile Movement:** the projectile movement component driving the arrow's trajectory (a gravity-affected ballistic path)
- **Hit Collision:** the arrow's hit collider, triggering `OnComponentBeginOverlap` for hit detection

3.2 Launch and Hit Logic

Purpose

Implements the arrow's launch control and hit detection. The arrow does not fly immediately on spawn; it waits for the bow's ArrowShoot command before starting ProjectileMovement. On hitting an object it immediately stops and destroys after a delay, while the Is Shoot? flag and a DoOnce node guarantee only one valid hit is processed, avoiding false triggers while attached to the hand or double scoring when penetrating multiple objects.

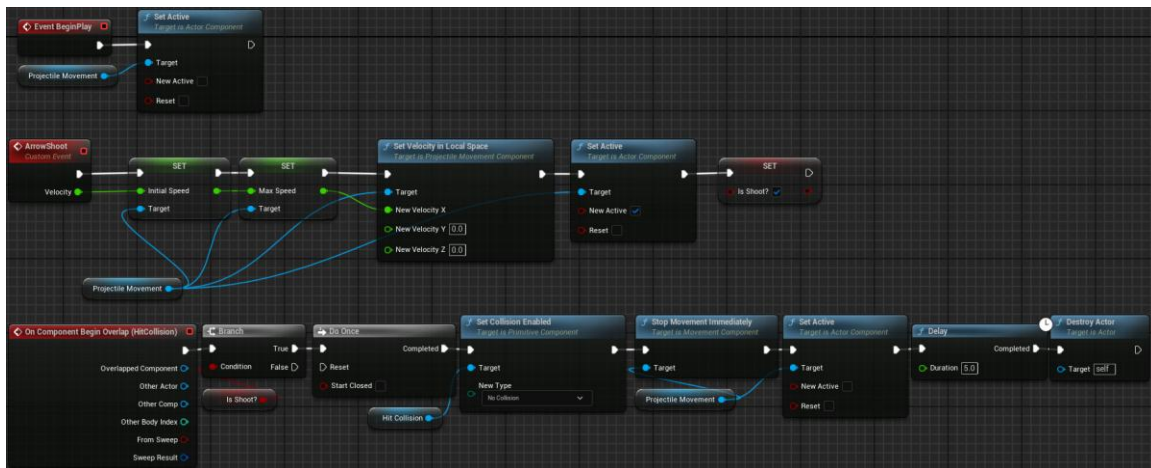
ArrowShoot (custom event)

Moves the arrow from "standby" into "flight" : receives the launch speed from the bow and starts flying.

OnComponentBeginOverlap (HitCollision)

Detects the first object the arrow meets in flight and performs the "hit freeze" — the arrow instantly stops at the hit point and is destroyed after a delay, with only one valid hit processed.

Blueprint screenshots



Key points

- Is Shoot? + Do Once double anti-false-trigger: Is Shoot? blocks Overlaps during the "attached to hand" phase, and Do Once blocks the second trigger after the hit penetrates into other objects — both are indispensable
- Three-step stop after a hit (disable collision → stop movement → stop component): disable collision first to block further Overlaps, then stop ProjectileMovement so the arrow pins at the hit position
- Destroyed after a 5-second delay: the arrow stays pinned on the target a moment before disappearing, giving the player hit feedback time

4. BP_Target_Bracket (Target)

4.1 Overview

Purpose

Implements the archery game's target — a multi-ring concentric target. Each ring is an independent static mesh, arranged front-to-back without overlapping, so an arrow hits

exactly one ring. Each ring has its own `OnComponentBeginOverlap` event, and hitting different rings scores differently (ring 10 = 10 points, ring 9 = 9 points, and so on).

4.2 Multi-Ring Hit Detection

Purpose

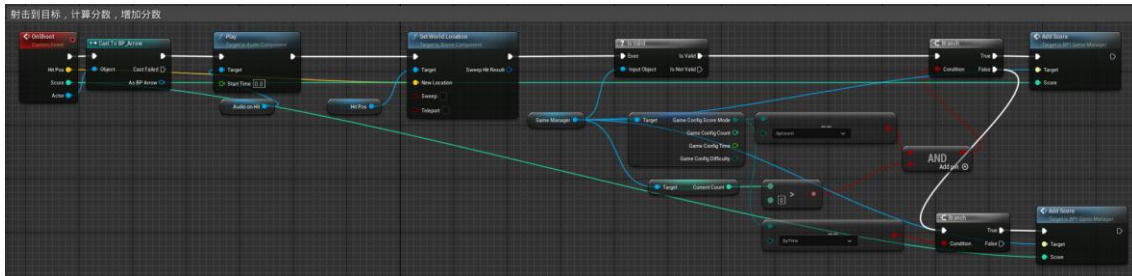
Leverages the structure of "each ring is an independent static mesh": each ring gets its own Overlap event and score, so hitting different rings scores differently. Per-ring detection removes the math of computing the hit point's distance to the center — the structure is intuitive and easy to extend: adding a ring just means copying the event and changing the score.

Each ring's OnComponentBeginOverlap

Every ring component has its own Overlap event, all structurally identical with only the score differing:

- SM_Ring_10 → Score = 10
- SM_Ring_9 → Score = 9
- SM_Ring_8 → Score = 8
- SM_Ring_7 → Score = 7
- (and so on for the remaining rings)
- Each event calls OnShoot with: HitPos = Get Actor Location (the target's center), Score = the ring's score, Actor = the arrow that hit

Blueprint screenshots



Key points

- Every ring must enable Generate Overlap Events and use Query collision: static meshes may not generate Overlap events by default, so confirm in the component details
- Scores are hardcoded directly in each ring's event: repetitive but simple and direct

4.3 OnShoot — Hit Handling and Scoring

Purpose

Handles the full feedback flow after an arrow hits the target: validate the hitting object, play the hit sound, position the arrow at the target's center (visually resting at the

bullseye), and decide whether the score is valid per the GameManager's mode (countdown/attempt) and current game state. As the "hit target", the target only reports the score; whether it counts is decided uniformly by the GameManager.

Blueprint screenshots

